

Αρχές Γλωσσών Προγραμματισμού & Μεταφραστών			
Τμήμα Μηχανικών Η/Υ & Πληροφορικής - Πανεπιστήμιο Πατρών			
Εαρινό Εξάμηνο 2026			
Εργαστηριακή Άσκηση Python — Συλλογή & Ανάλυση Εκπαιδευτικών Μαθημάτων			
Ονοματεπώνυμο	ΑΜ	Έτος	e-mail
Κάνδηλας Γιώργος	1115510	2ο	up1115510@ac.upatras.gr
Καρπέτας Απόστολος	1115507	2ο	up1115507@ac.upatras.gr

1. Εισαγωγή

Η εργασία υλοποιεί μια πλήρη desktop εφαρμογή σε Python με γραφικό περιβάλλον Tkinter, που συλλέγει εκπαιδευτικά μαθήματα (courses) από έξι online πηγές: τρεις μέσω δημόσιων API και τρεις μέσω Web Scraping. Τα δεδομένα αποθηκεύονται τοπικά σε αρχείο CSV, εμφανίζονται σε πίνακα με δυνατότητα φιλτραρίσματος, οπτικοποιούνται με τρία Matplotlib γραφήματα, και τροφοδοτούν μια μηχανή σύστασης που επιστρέφει top-3 προτάσεις βάσει σύνθετης βαθμολογίας. Επιπλέον, υλοποιήθηκε αυτόματος χρονοπρογραμματιστής (scheduler) που ανανεώνει τα δεδομένα στο background.

Το πρόγραμμα τρέχει ως:

```
python main.py
```

Κατά την εκτέλεση, κάθε πηγή δεδομένων εκτυπώνει στην κονσόλα μήνυμα της μορφής:

```
[Coursera-API] Status: Success - 20 μαθήματα
[iTunes-API] Status: Success - 15 μαθήματα
[GitHub-API] Status: Success - 15 μαθήματα
[W3Schools-Scraper] Status: Success - 12 μαθήματα
[YaleOCW-Scraper] Status: Success - 12 μαθήματα
[TutorialsPoint-Scraper] Status: Success - 12 μαθήματα
[Storage] Status: Success - 74 νέα, 74 σύνολο στο courses_1115510_1115507.csv
```

2. Αρχιτεκτονική & Δομή Κώδικα

Ο κώδικας χωρίζεται σε οκτώ αρχεία με σαφή διαχωρισμό ευθυνών:

Αρχείο	Σκοπός
main.py	Tkinter GUI orchestrator — δημιουργεί το παράθυρο, ενώνει όλα τα modules
collectors.py	6 πηγές δεδομένων (3 API + 3 Scraping) + fallback λίστες
normalizer.py	Mapping ανομοιογενών κατηγοριών/δυσκολιών σε ενιαίο σύστημα
storage.py	CSV load/save/export με append + dedupe μέσω pandas

recommender.py	Composite-score αλγόριθμος — επιστρέφει top-3 προτάσεις
charts.py	3 Matplotlib γραφήματα (bar, pie, line)
scheduler.py	Background daemon thread για auto-refresh κάθε 6 ώρες
config.py	Κεντρικές σταθερές, βάρη αλγορίθμου, logger function

Εξαρτήσεις μεταξύ modules:

```

main.py
├── collectors.py (collect_all)
│   ├── normalizer.py (normalize_*)
│   └── config.py (log)
├── storage.py (load_csv, save_courses, export_csv)
├── recommender.py (recommend)
├── charts.py (bar_top_duration, pie_difficulty, line_cost_duration)
├── scheduler.py (AutoRefresher)
└── config.py (σταθερές)

```

3. Βιβλιοθήκες & Εργαλεία

Βιβλιοθήκη	Χρήση
requests	HTTP requests για τα 3 API (Coursera, iTunes, GitHub) και τους 3 web scrapers
beautifulsoup4	HTML parsing — εξαγωγή δεδομένων από W3Schools, Yale OCW, TutorialsPoint
pandas	DataFrame για CSV load/save, deduplicate, φιλτράρισμα, recommender calculations
matplotlib	3 στατιστικά γραφήματα: bar chart, pie chart, line plot
tkinter	Γραφικό περιβάλλον χρήστη (stdlib — δεν χρειάζεται εγκατάσταση)
threading	Background threads για συλλογή δεδομένων και auto-refresh (stdlib)
re	Regex για εξαγωγή αριθμών από κείμενο workload (stdlib)

Εγκατάσταση εξωτερικών βιβλιοθηκών:

```
pip install requests beautifulsoup4 pandas matplotlib
```

4. Συλλογή Δεδομένων

Η συλλογή ξεκινά από το κουμπί "Συλλογή Δεδομένων" στο GUI. Ο κώδικας εκτελείται σε background thread ώστε το παράθυρο να μην παγώνει. Κάθε πηγή υλοποιείται ως ξεχωριστή function και επιστρέφει λίστα από course dicts.

Όλα τα courses κατασκευάζονται μέσω κεντρικής helper function που καλεί αυτόματα `normalize_category` και `normalize_difficulty`:

```
# collectors.py
```

```
def _course(title, provider, category, difficulty, cost, duration, language,
source):
    return {
        'title': str(title).strip(),
        'provider': str(provider).strip(),
        'category': normalize_category(category), # αυτόματο mapping
        'difficulty': normalize_difficulty(difficulty),
        'cost': float(cost) if cost else 0.0,
        'duration_hours': float(duration) if duration else 0.0,
        'language': normalize_language(language),
        'source': source,
        'fetched_at': datetime.now().isoformat()
    }
```

4.1 API 1: Coursera

Χρησιμοποιείται το δημόσιο REST endpoint `courses.v1` (χωρίς authentication). Επιστρέφει μέχρι 20 μαθήματα με πεδία `name` και `workload`. Η διάρκεια εξάγεται από κείμενο τύπου `"5-10 hours/week"` με regex.

Παραδοχή: Το `category` και το `difficulty` αφήνονται hardcoded (`"programming"` / `"intermediate"`) γιατί το βασικό endpoint δεν επιστρέφει αυτά τα πεδία χωρίς επιπλέον authenticated calls.

```
# collectors.py
def fetch_coursera():
    url = 'https://api.coursera.org/api/courses.v1'
    params = {'fields': 'name,workload,primaryLanguages', 'limit': 20}
    try:
        r = requests.get(url, params=params, timeout=10, headers=HEADERS)
        r.raise_for_status()
        items = r.json().get('elements', [])
        if not items:
            raise ValueError('empty response')
        courses = []
        for item in items:
            name = item.get('name', 'Unknown Course')
            workload = item.get('workload', '10')
            langs = item.get('primaryLanguages', ['en'])
            lang = 'English' if not langs or langs[0] != 'en' else langs[0]
            hours = _parse_hours(workload) # regex: εξάγει πρώτο αριθμό
            courses.append(_course(name, 'Coursera', 'programming',
                                   'intermediate', 0, hours, lang, 'Coursera-
API'))
        log('Coursera-API', 'Success', f'{len(courses)} μαθήματα')
        return courses
    except Exception as e:
        log('Coursera-API', 'Fallback', str(e))
        return _COURSE_API_FALLBACK # πραγματικά γνωστά courses
```

4.2 API 2: iTunes Search

Το iTunes δεν έχει dedicated courses API, οπότε χρησιμοποιείται το **Search API** με `entity=podcast` για εκπαιδευτικά podcasts. Τα educational podcasts έχουν τίτλο, πάροχο και διάρκεια. Η διάρκεια υπολογίζεται ως milliseconds / 3.600.000.

```
# collectors.py
def fetch_itunes():
    url = 'https://itunes.apple.com/search'
    params = {'term': 'programming tutorial course',
              'entity': 'podcast', 'limit': 20, 'country': 'US'}
```

```

try:
    r = requests.get(url, params=params, timeout=10)
    r.raise_for_status()
    results = r.json().get('results', [])
    courses = []
    for item in results[:15]:
        title = item.get('trackName', 'Unknown')
        prov = item.get('artistName', 'Unknown')
        genre = item.get('primaryGenreName', 'Education')
        price = float(item.get('collectionPrice') or 0)
        ms = item.get('trackTimeMillis') or 0
        # Μιλισεκόντια -> ώρες (proxy εκτίμηση διάρκειας podcast)
        hours = round(ms / 3_600_000, 1) if ms else 5.0
        courses.append(_course(title, prov, genre, 'beginner',
                                price, hours, 'English', 'iTunes-API'))
    log('iTunes-API', 'Success', f'{len(courses)} μαθήματα')
    return courses
except Exception as e:
    log('iTunes-API', 'Fallback', str(e))
    return _ITUNES_FALLBACK

```

4.3 API 3: GitHub Search

To GitHub Search API (60 req/ώρα χωρίς authentication) αναζητά repositories με topics 'course' και 'tutorial'. Δεν υπάρχει πεδίο 'course duration' - χρησιμοποιούνται τα stars ως μέγεθος υλικού (stars ÷ 500, max 200 ώρες).

```

# collectors.py
def fetch_github():
    url = 'https://api.github.com/search/repositories'
    params = {'q': 'topic:course topic:tutorial language:python',
              'sort': 'stars', 'per_page': 15}
    try:
        r = requests.get(url, params=params, timeout=10, headers=HEADERS)
        r.raise_for_status()
        items = r.json().get('items', [])
        courses = []
        for item in items:
            title = item.get('name', 'Unknown').replace('-', ' ').title()
            provider = item.get('owner', {}).get('login', 'GitHub')
            topics = item.get('topics', [])
            stars = item.get('stargazers_count', 0)
            # Stars ως proxy για μέγεθος: cap στα 200h για αποφυγή ακραίων τιμών
            hours = min(round(stars / 500), 200)
            courses.append(_course(title, provider, _github_category(topics),
                                    'intermediate', 0, hours, 'English', 'GitHub-API'))
        log('GitHub-API', 'Success', f'{len(courses)} μαθήματα')
        return courses
    except Exception as e:
        log('GitHub-API', 'Fallback', str(e))
        return _GITHUB_FALLBACK

```

4.4 Scraper 1: W3Schools

To w3schools.com έχει static HTML χωρίς anti-scraping. Ψάχνουμε links που περιέχουν 'Tutorial' στο navigation για να ανακτήσουμε τους τίτλους των tutorials ως course names.

```

# collectors.py
def scrape_w3schools():
    url = 'https://www.w3schools.com/'
    try:

```

```

r = requests.get(url, headers=HEADERS, timeout=10)
r.raise_for_status()
soup = BeautifulSoup(r.text, 'html.parser')
courses, seen = [], set()
# Ψάχνουμε συνδέσμους που περιέχουν 'Tutorial' στο nav
for link in soup.find_all('a', href=True):
    text = link.get_text(strip=True)
    href = link['href']
    if 'Tutorial' in text and href.startswith('/') and text not in seen:
        seen.add(text)
        cat = text.replace(' Tutorial', '').lower().strip()
        courses.append(_course(text, 'W3Schools', cat, 'beginner',
                                0, 10, 'English', 'W3Schools-Scraper'))
    if len(courses) >= 12:
        break
if not courses:
    raise ValueError('δεν βρέθηκαν tutorial links')
log('W3Schools-Scraper', 'Success', f'{len(courses)} μαθήματα')
return courses
except Exception as e:
    log('W3Schools-Scraper', 'Fallback', str(e))
return _W3SCHOOLS_FALLBACK

```

4.5 Scraper 2: Open Yale Courses

Αρχικά σχεδιάστηκε να χρησιμοποιηθεί το MIT OpenCourseWare, αλλά το ocw.mit.edu δεν δούλευε σωστά. Αντίστοιχα, το OpenLearn (The Open University) επέστρεφε 403. Επιλέχθηκε το oyc.yale.edu (Open Yale Courses) που εν τέλη δούλεψε.

```

# collectors.py
def scrape_open_yale():
    url = 'https://oyc.yale.edu/courses'
    try:
        r = requests.get(url, headers=HEADERS, timeout=15)
        r.raise_for_status()
        soup = BeautifulSoup(r.text, 'html.parser')
        courses, seen = [], set()
        # Strategy 1: headings μέσα στο course listing
        for el in soup.find_all(['h3', 'h4', 'h2']):
            text = el.get_text(strip=True)
            skip =
['yale', 'open', 'search', 'home', 'about', 'contact', 'menu', 'login']
            if len(text) > 8 and text not in seen and not any(w in text.lower()
for w in skip):
                parent = el.find_parent('a') or el.find('a')
                dept = parent['href'].strip('/').split('/')[0] if parent else
'other'
                seen.add(text)
                courses.append(_course(text, 'Yale University', dept.replace('-',
', ' '),
                                'intermediate', 0, 25, 'English', 'YaleOCW-
Scraper'))
            if len(courses) >= 12: break
        if not courses:
            raise ValueError('δεν βρέθηκαν courses')
        log('YaleOCW-Scraper', 'Success', f'{len(courses)} μαθήματα')
        return courses
    except Exception as e:
        log('YaleOCW-Scraper', 'Fallback', str(e))
        return _YALE_FALLBACK

```

4.6 Scraper 3: TutorialsPoint

Το [tutorialspoint.com/tutorialslibrary.htm](https://www.tutorialspoint.com/tutorialslibrary.htm) παρέχει links με μορφή `/topic/index.htm`. Εξάγονται αυτά τα links για να δημιουργηθούν course entries με τον τίτλο κάθε tutorial.

```
# collectors.py
def scrape_tutorialspoint():
    url = 'https://www.tutorialspoint.com/tutorialslibrary.htm'
    try:
        r = requests.get(url, headers=HEADERS, timeout=12)
        r.raise_for_status()
        soup = BeautifulSoup(r.text, 'html.parser')
        courses, seen = [], set()
        # Τα tutorials έχουν links με μορφή /topic/index.htm
        for link in soup.find_all('a', href=True):
            href = link.get('href', '')
            text = link.get_text(strip=True)
            if '/index.htm' in href and len(text) > 3 and text not in seen:
                seen.add(text)
                courses.append(_course(f'{text} Tutorial', 'TutorialsPoint',
                                      text.lower(), 'beginner', 0, 12, 'English',
                                      'TutorialsPoint-Scraper'))
            if len(courses) >= 12: break
        log('TutorialsPoint-Scraper', 'Success', f'{len(courses)} μαθήματα')
        return courses
    except Exception as e:
        log('TutorialsPoint-Scraper', 'Fallback', str(e))
        return _TUTORIALSPPOINT_FALLBACK
```

Fallback λίστες: κάθε πηγή διαθέτει λίστα από πραγματικά γνωστά courses, ώστε η εφαρμογή να έχει δεδομένα ακόμα και αν το δίκτυο αποτύχει

5. Κανονικοποίηση Δεδομένων

Οι έξι πηγές επιστρέφουν categories με διαφορετική ορολογία (π.χ. "machine learning", "machine-learning", "ML", "Artificial Intelligence"). Το `normalizer.py` τις αντιστοιχίζει σε 7 κανονικοποιημένες τιμές: Programming, Data Science, Web Development, Mathematics, Business, Languages, Other.

Στρατηγική 2 σταδίων για κάθε lookup:

- Στάδιο 1 - Exact match: αν η raw τιμή βρίσκεται ακριβώς στον CATEGORY_MAP, επιστρέφεται αμέσως.
- Στάδιο 2 - Substring fallback: αν κάποιο keyword του map περιέχεται μέσα στην raw τιμή, επιστρέφεται η αντίστοιχη normalized τιμή. Αντιμετωπίζει τιμές τύπου "Introduction to Machine Learning" που δεν υπάρχουν ακριβώς αλλά περιέχουν γνωστό keyword.
- Default: "Other" / "Beginner"

```
# normalizer.py - παράδειγμα του CATEGORY_MAP (απόσπασμα)
CATEGORY_MAP = {
    'programming': 'Programming',
    'python': 'Programming',
    'algorithms': 'Programming',
    'data structures': 'Programming',
    'machine learning': 'Data Science',
    'deep learning': 'Data Science',
    'data science': 'Data Science',
}
```

```

    'web development': 'Web Development',
    'html':           'Web Development',
    'javascript':     'Web Development',
    'mathematics':    'Mathematics',
    'business':       'Business',
    'english':        'Languages',
    # ... (30+ εγγραφές συνολικά)
}

def normalize_category(raw):
    if not raw:
        return 'Other'
    key = str(raw).lower().strip()
    if key in CATEGORY_MAP:
        return CATEGORY_MAP[key]      # exact match
    for k, v in CATEGORY_MAP.items():
        if k in key:                  # substring fallback
            return v
    return 'Other'                    # default

```

Αντίστοιχα για δυσκολία - DIFFICULTY_MAP αντιστοιχίζει τις παραλλαγές (beginner, introductory, basic, easy, novice, ...) στις τιμές Beginner / Intermediate / Advanced. Υπάρχει και `normalize_language` για τη γλώσσα (English, Greek, French, Spanish, German).

6. Αποθήκευση & Διαχείριση CSV

Το αρχείο CSV ονομάζεται `courses_1115510_1115507.csv`. Διαχειρίζεται αποκλειστικά από το `storage.py`.

```

# storage.py
def save_courses(new_courses):
    if not new_courses:
        return
    df_old = load_csv()                # φόρτωσε τα υπάρχοντα
    df_new = pd.DataFrame(new_courses)
    # Append: συνδύασε παλιά + νέα
    df_combined = pd.concat([df_old, df_new],
        ignore_index=True).infer_objects(copy=False)
    # remove duplicates: κρατάμε το τελευταίο (πιο πρόσφατο) για κάθε (title,
    provider)
    df_combined = df_combined.drop_duplicates(subset=['title', 'provider'],
        keep='last')
    df_combined.to_csv(CSV_FILE, index=False)
    log('Storage', 'Success',
        f'{len(df_new)} νέα, {len(df_combined)} σύνολο στο {CSV_FILE}')

```

Βασικές αποφάσεις σχεδιασμού:

- **Append mode:** φορτώνουμε τα παλιά, κάνουμε `concat` με τα νέα.
- **Remove Duplicates:** σε (title, provider): ο συνδυασμός αναγνωρίζει μοναδικά ένα course. Μόνο τίτλος δεν αρκεί — π.χ. 'Python Tutorial' υπάρχει από W3Schools ΚΑΙ TutorialsPoint.
- **keep='last':** η πιο πρόσφατη fetch αντικαθιστά την παλαιότερη έκδοχή.
- **infer_objects(copy=False):** αποφεύγει FutureWarning του pandas.

7. Γραφικό Περιβάλλον (Tkinter)

Το GUI υλοποιείται στο main.py ως κλάση App(tk.Tk):

```
# main.py
class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(f'Courses App - {STUDENTS_NAME} ({AM})')
        # => 'Courses App - Κανδής Γιώργος, Καρπέτας Απόστολος (1115510,
        1115507)'
```

Τα βασικά τμήματα του UI:

- Toolbar: κουμπιά Συλλογή Δεδομένων, Ανανέωση Πίνακα, Εξαγωγή CSV, Bar/Pie/Line Chart, checkbox αυτόματης ανανέωσης.
- Φίλτρα: dropdowns Κατηγορία/Δυσκολία και text entry Μέγιστο Κόστος + κουμπιά Φιλτράρισε/Επαναφορά.
- Πίνακας: ttk.Treeview με στήλες title, provider, category, difficulty, cost, duration_hours, language, source. Υποστηρίζει scroll.
- Status bar: εμφανίζει μηνύματα προόδου (πλήθος μαθημάτων, αποτέλεσμα φιλτραρίσματος).
- Μηχανή Συστάσεων: LabelFrame με dropdowns Κατηγορία/Δυσκολία/Γλώσσα + entry Μέγιστο Κόστος + Text widget για αποτελέσματα.

Σημαντική λεπτομέρεια: η συλλογή δεδομένων τρέχει σε background thread για να μην παγώνει το GUI, και το UI update γίνεται μέσω self.after(0, callback):

```
# main.py
def _collect(self):
    self.status_var.set('Συλλογή δεδομένων σε εξέλιξη...')
    def worker():
        courses = collect_all()          # αυτό τρέχει σε background thread
        save_courses(courses)
        self.after(0, self._refresh_table) # UI update στο main thread
    threading.Thread(target=worker, daemon=True).start()
```

8. Μηχανή Έξυπνων Συστάσεων

Η μηχανή (recommender.py) λειτουργεί αποκλειστικά πάνω στο ιστορικό CSV (offline mode). Επιστρέφει τα 3 καλύτερα μαθήματα βάσει composite score.

8.1 Αλγόριθμος Composite Score

Βήματα:

1. Φιλτράρισμα: εφαρμόζονται τα κριτήρια category, difficulty, language, max_cost.
2. Έλεγχος ελλειπών τιμών: αν λείπει duration_hours σε κάποιο course, αντικαθίσταται με το median των υπόλοιπων.
3. Min-max normalization: duration και cost μπαίνουν στο [0, 1] ώστε να είναι συγκρίσιμα.
4. Composite score: $score = 0.6 \times norm_duration + 0.4 \times (1 - norm_cost)$
5. Επιστροφή top 3 με nlargest(3, 'score').

```
# recommender.py
from config import W_DURATION, W_COST      # 0.6 και 0.4

def recommend(df, category='All', difficulty='All', language='All',
max_cost=None):
    if df.empty:
        return []
    filtered = df.copy()
    # ... εφαρμογή φίλτρων ...

    # Ελλειψείς τιμές duration: αντικατάσταση με median
    median_dur = filtered['dur_num'].median()
    filtered['dur_num'] = filtered['dur_num'].fillna(
        median_dur if not pd.isna(median_dur) else 0)

    # Min-max normalization
    def norm(val, mn, mx):
        if mx == mn: return 0.5      # ουδέτερο αν όλα ίδια
        return (val - mn) / (mx - mn)

    norm_dur = (filtered['dur_num'] - dur_min) / (dur_max - dur_min)
    norm_cost = (filtered['cost_num'] - cost_min) / (cost_max - cost_min)

    # Composite: μεγαλύτερη διάρκεια = υψηλό score
    #               χαμηλότερο κόστος = υψηλό score (1 - norm_cost)
    filtered['score'] = W_DURATION * norm_dur + W_COST * (1 - norm_cost)

    top3 = filtered.nlargest(3, 'score')
    return top3[result_cols].to_dict('records')
```

8.2 Αιτιολόγηση Βαρών

Τα βάρη ορίζονται στο config.py:

- $W_DURATION = 0.6$: Μαθήματα με περισσότερες ώρες παρέχουν πιο πλούσιο εκπαιδευτικό περιεχόμενο, άρα είναι πιο αξιολογικά ανεξαρτήτως κόστους.
- $W_COST = 0.4$: Το κόστος διαδραματίζει δευτερεύοντα ρόλο — μεταξύ courses παρόμοιας διάρκειας, προτιμώνται τα φθηνά/δωρεάν.
- Άθροισμα 1.0: Εξασφαλίζει ότι το score ανήκει στο $[0, 1]$ και είναι εύκολα ερμηνεύσιμο.

9. Γραφήματα (Matplotlib)

9.1 Bar Chart - Top 5 Μαθήματα σε Διάρκεια

Εμφανίζει τα 5 μαθήματα με τη μεγαλύτερη duration_hours. Οι τίτλοι κόβονται στους 30 χαρακτήρες για να χωράνε στον άξονα X.

```
# charts.py
def bar_top_duration(df):
    df_copy = df.copy()
    df_copy['duration_hours'] = pd.to_numeric(df_copy['duration_hours'],
                                              errors='coerce').fillna(0)
    top5 = df_copy.nlargest(5, 'duration_hours')
    fig, ax = plt.subplots(figsize=(9, 5))
    labels = [t[:30] + '...' if len(str(t)) > 30 else str(t)
              for t in top5['title']]
    ax.bar(labels, top5['duration_hours'], color='steelblue')
    ax.set_xlabel('Μάθημα')
```

```
ax.set_ylabel('Διάρκεια (ώρες)')
ax.set_title('Top 5 Μαθήματα με Μεγαλύτερη Διάρκεια')
plt.xticks(rotation=20, ha='right', fontsize=9)
plt.tight_layout()
plt.show()
```

9.2 Pie Chart - Κατανομή Επιπέδου Δυσκολίας

Εμφανίζει την κατανομή όλων των συλλεχθέντων μαθημάτων ανά επίπεδο δυσκολίας (Beginner/Intermediate/Advanced) με ποσοστά.

```
# charts.py
def pie_difficulty(df):
    counts = df['difficulty'].value_counts()
    fig, ax = plt.subplots(figsize=(7, 7))
    ax.pie(counts.values, labels=counts.index, autopct='%1.1f%%',
           startangle=140, colors=['#4CAF50', '#FF9800', '#F44336'])
    ax.set_title('Κατανομή Επιπέδου Δυσκολίας')
    plt.tight_layout()
    plt.show()
```

9.3 Line Plot - Συσχέτιση Κόστους και Διάρκειας

Εμφανίζει τη σχέση κόστους-διάρκειας για τα 5 μαθήματα με τη μεγαλύτερη διάρκεια. Τα δεδομένα ταξινομούνται κατά αύξουσα διάρκεια πριν σχεδιαστεί η γραμμή, ώστε να μην είναι 'τσαλακωμένη'.

```
# charts.py
def line_cost_duration(df):
    df_copy = df.copy()
    df_copy['duration_hours'] = pd.to_numeric(df_copy['duration_hours'],
                                              errors='coerce').fillna(0)
    df_copy['cost'] = pd.to_numeric(df_copy['cost'], errors='coerce').fillna(0)
    # Top 5, ταξινομημένα κατά αύξουσα διάρκεια για σωστή σχεδίαση γραμμής
    top5 = df_copy.nlargest(5, 'duration_hours').sort_values('duration_hours')
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(top5['duration_hours'], top5['cost'],
            marker='o', color='darkorange', linewidth=2, label='Κόστος')
    for _, row in top5.iterrows():
        ax.annotate(str(row['title'])[:20], (row['duration_hours'],
        row['cost']),
                    textcoords='offset points', xytext=(5, 5), fontsize=8)
    ax.set_xlabel('Διάρκεια (ώρες)')
    ax.set_ylabel('Κόστος (euros)')
    ax.set_title('Σχέση Κόστους και Διάρκειας - Top 5 Μαθήματα')
    ax.legend()
    plt.tight_layout()
    plt.show()
```

10. Χρονοπρογραμματιστής (Scheduler)

Η κλάση AutoRefresher (scheduler.py) εκτελεί αυτόματη ανανέωση κάθε 6 ώρες (ρυθμιζόμενο από SCHEDULER_INTERVAL_HOURS στο config.py), χωρίς χειροκίνητη παρέμβαση.

```
# scheduler.py
class AutoRefresher:
    def __init__(self, collect_fn, refresh_fn, interval_hours=None):
        self.collect_fn = collect_fn
```

```

self.refresh_fn = refresh_fn
self.interval = (interval_hours or SCHEDULER_INTERVAL_HOURS) * 3600
self._thread = None
self._active = False

def start(self):
    if self._active: return
    self._active = True
    # daemon=True: ο thread τερματίζεται όταν κλείσει το παράθυρο
    self._thread = threading.Thread(target=self._loop, daemon=True)
    self._thread.start()
    log('Scheduler', 'Started', f'ανανέωση ανά {self.interval // 3600}
ώρες')

def _loop(self):
    while self._active:
        time.sleep(self.interval)
        if self._active:
            log('Scheduler', 'Running', 'αυτόματη ανανέωση ξεκινά')
            courses = self.collect_fn()
            save_courses(courses)
            self.refresh_fn() # ανανεώνει τον πίνακα στο GUI

```

Βασικές αποφάσεις:

- `daemon=True`: ο thread τερματίζεται αυτόματα με το κύριο παράθυρο - δεν χρειάζεται ρητή διαχείριση κύκλου ζωής.
- `time.sleep` αντί `schedule library`: απλούστερο, χωρίς εξωτερική εξάρτηση. Ο `interval` από `config` μετατρέπεται εσωτερικά.
- Ο χρήστης ενεργοποιεί/απενεργοποιεί τον scheduler από το checkbox 'Αυτόματη ανανέωση (6h)' στην toolbar.

11. Στιγμιότυπα Εκτέλεσης

Παρακάτω παρουσιάζονται screenshots της εφαρμογής σε λειτουργία.

[Στιγμιότυπο 1 - Κύριο παράθυρο μετά από Συλλογή Δεδομένων: πίνακας με όλα τα courses]

[Στιγμιότυπο 2 - Εφαρμογή φίλτρου κατηγορίας 'Data Science']

[Στιγμιότυπο 3 - Bar Chart: Top 5 μαθήματα με μεγαλύτερη διάρκεια]

[Στιγμιότυπο 4 - Pie Chart: κατανομή επιπέδου δυσκολίας]

[Στιγμιότυπο 5 - Line Plot: συσχέτιση κόστους και διάρκειας]

[Στιγμιότυπο 6 - Μηχανή Συστάσεων: αποτέλεσμα top-3 για κριτήρια Programming / Beginner]

[Στιγμιότυπο 7 - Console output κατά τη συλλογή]

12. Σχόλια & Παραδοχές

Παραδοχή	Αιτιολόγηση
iTunes entity=podcast	Δεν υπάρχει courses API στο iTunes. Τα educational podcasts έχουν τίτλο, πάροχο και διάρκεια
MIT OCW → Open Yale Courses	Το ocw.mit.edu δεν δούλεψε. Αντικαταστάθηκε με το oyc.yale.edu (παλιό Drupal site).
Coursera category/difficulty hardcoded	Το Coursera API v1 χωρίς auth δεν επιστρέφει αυτά τα πεδία στο basic endpoint.
GitHub stars → duration proxy	Δεν υπάρχει πεδίο 'course duration' σε GitHub repo. Stars ÷ 500 ως 'μέγεθος'. Max στις 200 ώρες αποτρέπει ακραίες τιμές.
iTunes ms → ώρες	trackTimeMillis / 3.600.000 - ακριβής μετατροπή milliseconds σε ώρες.
Language = 'English' σε fallbacks	Οι πηγές που χρησιμοποιούμε παράγουν κυρίως αγγλόφωνο περιεχόμενο.
Median fill για ελλιπές duration	Αντί να τιμωρούμε courses χωρίς διάρκεια (τιμή 0) ή να τα αγνοούμε, βάζουμε το median.
time.sleep αντί schedule library	Απλούστερο.

13. Προβλήματα & Λύσεις

Πρόβλημα	Λύση
GUI παγώνει κατά τη συλλογή	Χρήση threading.Thread(daemon=True) για background execution. Το self.after(0, callback) εξασφαλίζει ότι το UI update γίνεται στο main thread (απαιτήση Tkinter).
MIT OCW και OpenLearn αδύνατο scraping	Το MIT OCW είναι React SPA, το OpenLearn έδινε 403. Μετάβαση στο oyc.yale.edu που είναι Drupal site με server-side rendering.
FutureWarning pandas κατά concat	Προσθήκη infer_objects(copy=False) μετά το concat - αποφεύγει deprecation warning χωρίς λογική αλλαγή.
Τίτλοι χαρτιών μεγάλοι για τον άξονα X	Κόψιμο τίτλων στους 30 χαρακτήρες + rotation=20° στα xtick labels.
Line chart με 'τσαλακωμένη' γραμμή	Προσθήκη sort_values('duration_hours') πριν το plot για αύξουσα σειρά στον άξονα X.
GitHub API rate limit	Το limit των 60 requests/ώρα χωρίς authentication είναι αρκετό. Σε περίπτωση εξάντλησης, ενεργοποιείται αυτόματα το fallback.
Coursera API επιστρέφει 'elements': []	Προσθήκη ελέγχου αν τα items είναι κενά και αυτόματη μετάβαση στο fallback.